Data Wrangling: Cleaning, Merging & GroupBy

Suman 18 Feb, 2026 At 8:00 PM Pre-Reads Module-1 Recommended Resource

Associated Lectures (1)

Description Discussions

Session 4.2: Pre-read Material

Data Wrangling: Cleaning, Merging & GroupBy

Program: Vishlesan i-Hub IIT Patna x Masai School -- AIM (AI & Machine Learning) **Session ID:** 4.2 | **Week:** 4 **Prerequisites:** Session 4.1 (Pandas DataFrames: Creation, Indexing & Selection)

What This Session Covers

In Session 4.1, you learned to create DataFrames, load CSV files, select rows and columns, filter with boolean conditions, and sort results. But you worked with **clean, well-structured data**. Real-world data is never clean. It arrives with missing values, duplicate entries, inconsistent formatting, wrong data types, and scattered across multiple files that need to be combined. This session teaches you to handle all of that.

The session covers eight topics:

- **Handling Missing Values** -- Detecting, counting, removing, and filling missing data using `.isna()`, `.dropna()`, `.fillna()` with context-appropriate strategies
- **Handling Duplicates** -- Finding and removing duplicate records with `.duplicated()` and `.drop_duplicates()`, including subset-based detection
- **Data Type Conversion** -- Fixing columns stored as the wrong type using `.astype()`, `pd.to_numeric()`, and `pd.to_datetime()` with error handling
- **String Operations** -- Cleaning inconsistent text (whitespace, capitalization, typos) using the `.str` accessor methods
- **Messy Data Patterns** -- Recognising tidy data principles, wide vs long format, and when to reshape data
- **Combining DataFrames: Concat** -- Stacking DataFrames vertically or horizontally using `pd.concat()`
- **Combining DataFrames: Merge/Join** -- Merging DataFrames using `pd.merge()` with inner, left, right, and outer joins

GroupBy Operations -- Performing split-apply-combine analysis using `.groupby()`, `.agg()`, `.transform()`, and `.pivot_table()`

By the end of this session, you will be able to take any messy dataset -- with missing values, duplicates, wrong types, and scattered across multiple files -- and transform it into a clean, merged, analysis-ready result with group-level summaries.

Prerequisites Checklist

Before attending this session, make sure you are comfortable with everything from Session 4.1:

- [ ] Creating DataFrames from dictionaries and loading CSV files with `pd.read_csv()`
- [ ] Exploring data with the **Five-Method Ritual**: `.head()`, `.tail()`, `.info()`, `.describe()`, `.shape`
- [ ] Selecting columns using bracket notation (`df['column']`) and dot notation (`df.column`)
- [ ] Selecting rows by label with `.loc[]` and by position with `.iloc[]`
- [ ] Filtering rows with boolean conditions: `df[df['column'] > value]`
- [ ] Sorting DataFrames with `.sort_values()`
- [ ] Understanding that NaN appeared when creating DataFrames from dictionaries with missing keys
- [ ] Understanding that `order_date` was stored as a string (`object` dtype) in Session 4.1 -- we deferred fixing it to this session

If any of these feel unclear, review your Session 4.1 materials before class.

Important note: This session picks up exactly where 4.1 left off. The NaN values you saw briefly and the `object` -typed date column you left unfixed -- both are addressed systematically in this session.

Business Story: Healthcare Data Integration Failures

Before you write a single line of data-cleaning code, you need to understand WHY this skill matters. The answer is not academic -- it comes from real failures in healthcare that affected real patients.

The Hospital Merger Problem

When hospitals merge or when patient records need to be combined across departments, data integration becomes a life-or-death challenge. Consider a typical large hospital with three separate systems:

- **Emergency Department** runs **System A** -- some intake form fields are optional (allergies, family history), and patient names are entered in `UPPER CASE`
- **Outpatient Clinic** runs **System B** -- different field names (System A calls it `patient_name`, System B calls it `full_name`), different required fields, and dates stored as `DD-MM-YYYY`
- **Lab Results** come from **System C** -- no patient name at all, only an ID number, and dates stored as `YYYY-MM-DD`

When these three systems are merged into a unified patient database, the result is chaos:

- **35% of records** had at least one missing field after the merge (a 2019 study found this rate across healthcare integration projects)
- **Duplicate patient records** proliferated -- the same patient appeared as "RAJESH KUMAR" (from Emergency), "Rajesh Kumar" (from Outpatient), and patient ID 4527 (from Lab) -- three separate records for one person
- **Date mismatches** caused scheduling errors -- an appointment on `01-02-2024` meant January 2nd in one system and February 1st in another
- **Billing duplicates** meant patients were charged twice for the same procedure, triggering fraud investigations

Real Consequences

These are not theoretical problems:

- **Wrong medications:** A patient's allergy record existed in one system but was missing after the merge. The merged record showed no allergies. A doctor prescribed a medication the patient was allergic to.
- **Duplicate billing fraud flags:** A hospital submitted what appeared to be duplicate insurance claims -- same patient, same procedure, same date -- because the records had not been deduplicated. The hospital was investigated for billing fraud.
- **Lost medical histories:** During a system migration, inconsistent date formats caused 12,000 records to be sorted incorrectly. Patients' chronological medical histories were scrambled, making it impossible to track disease progression.
- **Transfusion errors:** A missing blood type field -- left blank because it was optional in one system -- was not flagged during the merge. The absence of this critical information risked a transfusion mismatch.

Why This Matters for You

A 2019 industry study found that **86% of healthcare data integration projects experience significant data quality issues**. The root causes are exactly the problems you will learn to solve in this session: missing values, duplicates, inconsistent text formatting, wrong data types, and the need to merge data from multiple sources.

Data wrangling is not just a technical skill -- in domains like healthcare, finance, and safety, it is a matter of human welfare.

Think About:

- Have you ever filled out a form where some fields were optional? What happens to those blank fields in a database?
- If two hospital systems record dates differently (`DD-MM-YYYY` vs `YYYY-MM-DD`), how would you know which format a particular date uses?
- Why might a "duplicate" patient record not be caught by a simple exact-match check?

Key Concepts to Preview

Read through these previews carefully. You do not need to memorise every detail -- the goal is to arrive in class with a general sense of what each concept is, so the hands-on demonstrations feel familiar rather than foreign.

Missing Values: The Medical Form Analogy

Imagine a hospital intake form. A patient walks in, and the nurse fills out the form:

```
Patient Intake Form
------------------------------
Name:       Rajesh Kumar        <-- filled
Age:        45                  <-- filled
Blood Type: A+                  <-- filled
Allergy:    _________________   <-- BLANK
Weight:     _________________   <-- BLANK
```

Those blank fields are **missing values**. In Pandas, they are represented as **NaN** (Not a Number) -- a special marker that means "this value was not recorded."

Key things to know about NaN:

- **NaN is not zero.** A blank weight field does not mean the patient weighs 0 kg -- it means the weight was not measured.
- **NaN is not an empty string.** It is the complete absence of a value.
- **NaN is contagious in math:** `5 + NaN = NaN`. Any calculation involving NaN produces NaN.
- **NaN is not equal to itself:** `NaN == NaN` returns `False`. This is a deliberate design choice inherited from the IEEE floating-point standard.

The critical question for every missing value is: **should you remove the row, fill it with something, or leave it as-is?** The answer depends on context:

- A missing **blood type** in a hospital cannot be filled with the "average" blood type -- that is medically dangerous. You must flag it and investigate.
- A missing **weight** can reasonably be filled with an average for statistical analysis.
- A missing **spouse name** might genuinely mean the person is unmarried -- it is not really "missing" at all.

You will learn `.isna()` to detect missing values, `.dropna()` to remove rows with missing values, and `.fillna()` to replace missing values with a chosen strategy (constant, mean, median, or forward fill).

Duplicates: The Hospital Wristband Analogy

Imagine a patient arrives at the Emergency Department. Due to a system glitch, **two wristbands** are printed with the same patient ID. Now:

- The nurse scans wristband 1 and records vitals
- A different nurse scans wristband 2 and records vitals **again**
- The billing system counts **two admissions** -- the patient is billed twice
- The pharmacy system sees **two active patients** -- medication could be dispensed twice
- Your analytics report shows inflated patient counts and skewed averages

The dangerous thing about duplicates is that they do not cause errors. Your code runs perfectly. Your numbers look reasonable. But they are **silently wrong** because some records are counted more than once.

You will learn `.duplicated()` to detect duplicate rows and `.drop_duplicates()` to remove them. A key concept is the `subset` parameter -- you can check for duplicates based on a specific column (like `patient_id`) rather than requiring every single column to match exactly.

Data Type Conversion: The Currency Analogy

Consider the number `100`. As dollars, it buys a decent meal. As rupees, it buys a cup of tea. As yen, it barely buys a candy bar. The raw digits are identical, but the **type** (the interpretation) changes everything.

In Pandas, the same problem occurs with data types:

- A column containing `['15000', '22500', '8900']` **looks** numeric -- but if it is stored as strings (`object` dtype), you cannot do math on it. `'15000' + '22500'` gives `'1500022500'` (string concatenation), not `37500`.
- A column containing `['2024-01-15', '2024-01-16']` **looks** like dates -- but if stored as strings, you cannot sort chronologically or calculate days between dates.

One rogue value can ruin an entire column: if a billing column contains `['15000', '22500', 'N/A', '8900']`, Pandas stores the **entire column** as strings because of that single `'N/A'` entry. One bad apple spoils the bunch.

You will learn three conversion tools:

- **`.astype()`** -- for simple, clean conversions where you are confident every value is valid
- **`pd.to_numeric()`** -- for converting strings to numbers, with an `errors='coerce'` option that turns unconvertible values into NaN instead of crashing
- **`pd.to_datetime()`** -- for converting date strings to proper datetime objects, even when the column has mixed formats

String Operations: The Find-and-Replace Analogy

You know Find-and-Replace in a word processor -- you search for a misspelling and replace it everywhere in the document. The `.str` accessor in Pandas does the same thing, but for an **entire column at once**.

Consider a `department` column that contains `'Cardiology'`, `'cardiology'`, `'CARDIOLOGY'`, and `' cardiology '`. These are all the same department, but Pandas treats them as four different values because string comparison is case-sensitive and whitespace-sensitive. If you group by department, you get four separate groups instead of one.

The `.str` accessor gives you vectorised string methods -- they apply to every value in a column simultaneously, no loops needed:

Method	What It Does	Example
`.str.strip()`	Removes leading/trailing whitespace	`' Priya '` becomes `'Priya'`

Method	What It Does	Example
`.str.lower()`	Converts to lowercase	`'ANITA'` becomes `'anita'`
`.str.upper()`	Converts to uppercase	`'anita'` becomes `'ANITA'`
`.str.title()`	Converts to Title Case	`'vikram patel'` becomes `'Vikram Patel'`
`.str.replace(old, new)`	Replaces substrings	Targeted text replacement
`.str.contains(pattern)`	Returns True/False for substring match	Filtering rows by text content

A key advantage: `.str` methods automatically **skip NaN values** instead of crashing on them.

The standard cleaning pipeline is: **strip whitespace first, then standardise case**. This can be chained in a single line: `df['name'] = df['name'].str.strip().str.title()`.

Tidy Data: The Filing Cabinet Analogy

In 2014, statistician Hadley Wickham formalised three rules that define **tidy data**:

> **Each variable forms a column** -- one piece of information per column
> **Each observation forms a row** -- one entity or event per row
> **Each type of observational unit forms a table** -- customers in one table, orders in another

Think of it as the difference between a **well-organised filing cabinet** (each drawer is labelled, each folder is one person's file, separate cabinets for HR, Finance, Sales) and a **drawer full of loose papers** (everything mixed together, you dig through the pile every time).

A common violation is **wide format** -- where column headers contain data values instead of variable names:

```
WIDE (messy):
| Student | Math | Science | English |
|---------|------|---------|---------|
| Priya   |  85  |   92    |   78    |

LONG/TIDY (clean):
| Student | Subject | Score |
|---------|---------|-------|
| Priya   | Math    |  85   |
| Priya   | Science |  92   |
| Priya   | English |  78   |
```

In the wide format, "Math", "Science", and "English" are actually **values** of a variable called "Subject" -- they are masquerading as column names. In the tidy format, each row represents one observation (one student's score in one subject), and each column is a single variable.

You will learn to **recognise** messy patterns in this session. The tools to fix them (`pd.melt()` and `.pivot()`) will be introduced at an awareness level.

Concat: The Stacking Trays Analogy

Imagine a cafeteria with food trays. **Vertical stacking** (`pd.concat()` with `axis=0`) is putting one tray on top of another -- you get more items (rows), but the categories (columns) stay the same. This is the most common scenario: January sales in one file, February sales in another, and you stack them into a single table.

```
January (3 rows):         February (3 rows):
| order_id | amount |     | order_id | amount |
|----------|--------|     |----------|--------|
| ORD001   |  899   |     | ORD004   | 54999  |
| ORD002   | 3499   |     | ORD005   |   899  |
| ORD003   | 15999  |     | ORD006   |  1299  |

After pd.concat([jan, feb]):
| order_id | amount |
|----------|--------|
| ORD001   |  899   |
| ORD002   | 3499   |
| ORD003   | 15999  |
| ORD004   | 54999  |
| ORD005   |   899  |
| ORD006   |  1299  |
```

The key thing about concat is that it is "dumb gluing" -- it does not look at values to decide what goes where. It simply puts one DataFrame after the other. Always use `ignore_index=True` to avoid duplicate index values after stacking.

Merge: The VLOOKUP Analogy

If you have ever used **VLOOKUP in Excel**, you already understand the concept of merge. Your `orders` table has a `customer_id` column, but it does not have the customer's name or city -- that information lives in a separate `customers` table. You need to **look up** each customer_id in the customers table and pull back the matching name and city.

`pd.merge()` is VLOOKUP on steroids: no direction limits, no column-position dependency, brings all matching columns at once, and works on millions of rows.

```
orders:                        customers:
| order_id | customer_id |     | customer_id | name         | city      |
|----------|-------------|     |-------------|--------------|-----------|
| ORD001   | C001        | --> | C001        | Priya Sharma | Mumbai    |
| ORD002   | C002        | --> | C002        | Rahul Verma  | Delhi     |

Result: Each order now has customer name and city attached.
```

The `how` parameter controls which rows survive the merge:

- **Inner join** (`how='inner'`) -- only rows with matching keys in **both** tables survive. This is the default.
- **Left join** (`how='left'`) -- **all** rows from the left table are kept, even if there is no match in the right table (unmatched rows get NaN for the right table's columns). This is the most commonly used join in practice.

- **Right join** (`how='right'`) -- the mirror of left join, keeping all rows from the right table.
- **Outer join** (`how='outer'`) -- **all** rows from **both** tables are kept. Useful for finding what is missing.

GroupBy: The Sorting-Into-Houses Analogy

Imagine you are a teacher with 200 exam papers from 5 different sections. To calculate each section's average score, you would:

Split the papers into 5 piles -- one pile per section

Apply the "calculate average" operation to each pile

Combine the 5 averages into a summary table

This is the **split-apply-combine** pattern, and it is exactly what `.groupby()` does:

```
# "What are the total sales per city?"
df.groupby('city')['sales'].sum()
```

Pandas splits the DataFrame into groups (one group per unique city), applies `.sum()` to the `sales` column within each group, and combines the results into a summary.

You will also learn:

- **`.agg()`** -- for applying multiple aggregations at once (sum AND mean AND count)
- **`.transform()`** -- for broadcasting a group-level result back to every row (e.g., adding a "city_total" column to each row so you can calculate each order's percentage of its city's total)
- **`.pivot_table()`** -- for creating cross-tabulation summaries (like Excel pivot tables)

Key Vocabulary

These terms will appear throughout the session. Familiarise yourself with them so they are not completely new when you hear them in class.

Term	Definition
NaN	"Not a Number" -- Pandas' universal marker for a missing or absent value. Comes from NumPy (`np.nan`). Not equal to itself.
Missing value	A cell in a DataFrame that contains NaN or None -- indicating the information was not recorded or is unavailable
`.isna()` / `.notna()`	Methods that return a Boolean mask showing where values are missing (True) or present (True), respectively
`.dropna()`	Removes rows (or columns) that contain missing values. The `subset` parameter targets specific columns.
`.fillna()`	Replaces missing values with a specified value (constant, mean, median, or forward/backward fill)

Term	Definition
Duplicate	A row that appears more than once in a dataset, either as an exact copy or matching on key columns
`.duplicated()`	Returns a Boolean mask marking duplicate rows. The `keep` parameter controls which occurrence is considered the "original."
Tidy data	A data format where each variable forms a column, each observation forms a row, and each type of unit forms a table
`pd.concat()`	Stacks DataFrames vertically (more rows) or horizontally (more columns). Does not match on keys -- simple "gluing."
`pd.merge()`	Combines two DataFrames by matching rows based on a shared key column. Equivalent to SQL JOIN or Excel VLOOKUP.
Inner join	A merge that keeps only rows with matching keys in both tables. No NaN introduced.
Left join	A merge that keeps all rows from the left table. Unmatched rows get NaN for right-table columns. Most common in practice.
Outer join	A merge that keeps all rows from both tables. Useful for finding what is missing in either table.
`.groupby()`	Splits a DataFrame into groups based on column values, enabling per-group aggregation (split-apply-combine).
`.agg()`	Applies one or more aggregation functions (sum, mean, count, etc.) to grouped data. Reduces rows.
`.transform()`	Applies a function to grouped data but returns a result with the same number of rows as the input (broadcasts group-level values back).
Pivot table	A cross-tabulation summary with one variable as rows, another as columns, and aggregated values in the cells

Things to Ponder

Come to class having thought about these questions. You do not need written answers, but engaging with them beforehand will help you follow the session more actively.

> **About missing data strategies:** You have a dataset of student exam scores for 200 students, and 40 of them (20%) have a missing score in the `midterm_marks` column. Would you drop those 40 rows, or fill the missing values? If you choose to fill, what value would you use -- zero, the class mean, the class median? Does the reason WHY the scores are missing affect your decision?

> **About duplicates hiding in plain sight:** If a customer database has two entries for "Priya Sharma" -- one recorded as `' Priya Sharma '` (with extra spaces) and the other as `'Priya Sharma'` -- would a simple equality check catch this as a duplicate? What preprocessing steps would you need to take first?

About merge strategy: You have an `orders` table with 1,000 rows and a `customers` table with 500 rows. If you use an inner join on `customer_id`, would the result have exactly 1,000 rows, exactly 500 rows, or some other number? What factors determine the result size?

About data types: If a column of bill amounts is stored as strings (`object` dtype) instead of numbers (`float64`), what operations would fail silently or produce wrong results? Think about sorting, calculating averages, and comparing values.

About tidy data: Consider a table where columns are `Student`, `Jan_Score`, `Feb_Score`, `Mar_Score`. Is this tidy or messy? How would you restructure it so that each row represents one student's score in one month?

Optional Deep Dive

If you want to explore further before or after class, these resources provide deeper context:

- **Pandas `pd.merge()` documentation:** https://pandas.pydata.org/docs/reference/api/pandas.merge.html -- comprehensive reference for all merge parameters and join types
- **Pandas `pd.concat()` documentation:** https://pandas.pydata.org/docs/reference/api/pandas.concat.html -- details on vertical/horizontal stacking and index handling
- **"Tidy Data" by Hadley Wickham (2014):** https://doi.org/10.18637/jss.v059.i10 -- the foundational paper that formalised the three tidy data rules
- **Pandas User Guide on Missing Data:** https://pandas.pydata.org/docs/user_guide/missing_data.html -- official guide on NaN handling, detection, and filling strategies
- **Pandas User Guide on GroupBy:** https://pandas.pydata.org/docs/user_guide/groupby.html -- split-apply-combine operations in depth
- **Pandas `.str` accessor documentation:** https://pandas.pydata.org/docs/user_guide/text.html -- full list of string methods available through the `.str` accessor

These are entirely optional. The session is self-contained and does not assume you have read any external material.

Pre-Class Checklist

Before you walk into class, make sure you can check off every item:

- [ ] I have Python 3.12.x installed with Jupyter Notebook (or JupyterLab)
- [ ] I can `import pandas as pd` and `import numpy as np` without errors
- [ ] I have read the Healthcare Data Integration Failures story and understand why data wrangling matters
- [ ] I understand what NaN means and that it represents a missing value (not zero, not an empty string)
- [ ] I know from Session 4.1 that `.info()` shows non-null counts per column -- this is how you spot missing data

- [ ] I understand the basic idea behind merging: looking up information from one table using a key column shared with another table
- [ ] I understand the split-apply-combine concept: split data into groups, calculate something per group, combine results
- [ ] I have reviewed the vocabulary table above and recognise the key terms
- [ ] I have thought about at least two of the "Things to Ponder" questions

---

Looking Ahead

After this session, you will be able to:

- Detect and handle missing values with appropriate strategies (drop, fill, or flag)
- Find and remove duplicate records, even when duplicates are hidden by whitespace or casing differences
- Convert columns to their correct data types so that numbers behave like numbers and dates behave like dates
- Clean messy text data with the `.str` accessor in a single line of code
- Stack DataFrames from multiple files using `pd.concat()`
- Merge DataFrames from different sources using `pd.merge()` with the right join type for your use case
- Summarise data with `.groupby()` to answer questions like "total sales per city" or "average order value per segment"
- Build a complete data wrangling pipeline: clean, merge, analyse, summarise

This session completes your Pandas data wrangling toolkit and sets the foundation for **Session 4.3**, which examines data quality from a broader perspective -- the principles, frameworks, and real-world disasters (IBM Watson Oncology, JPMorgan London Whale, UK Post Office Horizon) that happen when data quality is ignored.

See you in class!

Vishlesan i-Hub IIT Patna x Masai School